Governing Changes in Self-Evolving Agents

Majid Babaei

***Abstract*—Self-evolving agents improve themselves by rewriting their own prompts, tools, memory, and workflows. Passing a test does not keep a change authorized as the harness continues to evolve: another change may alter a component that the approved one relies on, or an operator may withdraw a faulty version from which other tools, skills, or memories were derived. We present SSEVOH-DSL, a domain-specific language (DSL) and trusted enforcement service for *Foundational Contracts*: operator-controlled rules that govern self-modifications and the continued use of their results. An approval is bound to the state, policy, and validator evidence it checked. The service then rechecks freshness at commit, artifact identity and eligibility at execution, and recorded origins at reuse. Withdrawing a version therefore blocks everything derived from it, and rollback restores only versions that remain eligible. The service governs which changes take effect without altering how the agent proposes them. Bounded checks over 248,832 one-step cases and 19,620 explored transitions, and TLA+ model checking, found no disagreement with independent reference semantics. We integrated the service into Harness^2, a self-evolving agent harness recently proposed by Google Research. Across deterministic, model-generated, and in-host tests, it blocked all 178 assessed contract violations, rejected no permitted change, and had a median admission time of 0.6 ms.**

***Index Terms*—Agent harnesses, self-evolving agents, domain-specific languages, runtime verification, authorization, provenance, model checking.**

I. Introduction

APPROVING a self-modification once is insufficient to keep a self-evolving agent's later actions authorized. A tested, approved change may become ineligible when its dependencies change or an operator withdraws a version from which it was derived. Approval must therefore remain tied to the context that justified it. This challenge grows as agents search over programs [1], [2], refine persistent context [3], and revise their execution harnesses [4], [5]. Their retained artifacts become the software governing future behavior. Controlling the release, continued use, and retirement of these artifacts is a software engineering requirement for self-evolving agents.

Consider an agent that repairs a ledger tool, builds a reporting skill using the repaired version, and stores a lesson for future tasks. Tests may show that the repair produces correct totals on the tested ledgers. However, these tests do not establish that the agent has permission to make the change or that its approval remains valid after another update. If the operator later withdraws the repair, the skill and stored lesson may still depend on it. Restoring the old tool does not remove those dependencies: the system must also prevent the affected skill and lesson from being used again.

Empirical studies reinforce this concern. Harness-tampering audits find integrity violations that persist through self-improvement lineages [6], while skill-composition attacks carry malicious influence across sessions [7]. Task success alone is therefore an incomplete release criterion. Useful changes may violate policy or inherit an invalid origin, while rejecting every unfamiliar implementation can prevent legitimate repairs.

Recent work governs parts of this lifecycle: promotion and rollback of generated artifacts [8], recovery [9], and authority across agent replacement and rollback [10]. We build on this perspective with *Foundational Contracts*: explicit rules connecting the permission to change a component, the evidence supporting that change, and the permission to keep using its results. Our contribution is a specified and evaluated combination of these obligations, in which each approval stays tied to the context that justified it and each withdrawal reaches everything derived from the withdrawn version, across approval, execution, recovery, and persistent reuse.

We present SSEVOH-DSL, a domain-specific language (DSL) and trusted service that enforce Foundational Contracts around an agent's improvement process. The agent proposes changes; the operator defines permissions and evidence requirements. Approval is specific to a version and context: a pending change needs fresh approval if that context changes. For active artifacts, the service prevents use of versions with withdrawn origins and temporarily disables dependents of replaced components until revalidated. The implementation agrees with independent semantics within checked bounds, and it rejects all 178 assessed violations across deterministic, model-generated, and in-host cohorts without rejecting a permitted change. Task studies expose a trade-off between evidence strength and usefulness but do not show an accuracy advantage over frozen selection.

Contributions. (1) Foundational Contracts for version-bound approval, dependency quarantine, transitive withdrawal, and eligible recovery. (2) A DSL, six enforcement algorithms, and explicit host-adapter obligations. (3) Independent bounded conformance, history/replay, and TLA+ model checks. (4) A case study measuring the contribution of contract clauses, integration fidelity, usefulness, and cost. Empirical portability beyond this host remains untested. Secs. II–VI present background, approach, evaluation, related work, and conclusions.

II. Background and Running Example

A. Self-evolving harnesses

Harness^2 [4] describes harness evolution through edits to the software surrounding a model and reuse of experience across tasks. ACE [3] similarly treats persistent context as an evolving artifact. Building on these accounts, we model a harness as named, typed, versioned components: tools, skills, memory, workflows, configuration, policy, and verifiers. We add two explicit relations: a *dependency* identifies a component needed for operation; a *parent* records a source used to derive a version. These relations let us distinguish changes that require retesting from withdrawn sources that disqualify later reuse.

B. Contracts in model-driven engineering

Design by Contract specifies obligations through preconditions, postconditions, and invariants [11]. In model-driven engineering (MDE), the Object Constraint Language (OCL) expresses such constraints over UML models [12]. Preconditions state when an operation may run, postconditions describe its required result, and invariants define valid system states. We adopt this discipline for a versioned harness model and add continued-use conditions: a version that satisfied its contract earlier may become ineligible after its context or origins change. We call these lifecycle obligations *Foundational Contracts* because they govern the infrastructure on which later agent behavior depends. Our DSL implements this specialization; it does not implement OCL or verify arbitrary generated programs.

C. Trust boundary and failure model

A reference monitor [13] forces every security-relevant action through an enforcement mechanism that the monitored party cannot alter. We apply this principle to self-modification: the agent may propose changes, but only trusted code decides whether they take effect, and the decision is rechecked whenever an artifact is activated, executed, or reused. The trusted parts are the service, operator policy, registered validators, and host adapter; agent proposals cannot modify them. We protect against edits the agent is not permitted to make, approvals made stale by a later change, conflicting updates, artifacts derived from withdrawn versions, and artifacts altered after approval. We assume that validators decide their predicates correctly, although finite tests cannot prove a program safe. We also assume that code the agent can change runs in a separate sandbox (Sec. IV-A), that hashes are collision-resistant, and that the journal's latest hash is stored externally so that wholesale replacement is detectable. We do not handle malicious validators, kernel compromise, actions that bypass the guards, or preserving state across restarts.

D. Running example: repairing a ledger tool

A ledger agent uses tool summarize and a dependent reporting skill, report. The tool's first version ignores refunds: for $9.00, -2.00$, it returns 9.00 instead of 7.00. The agent may repair the tool but may neither edit protected guardrails nor add network access. Listing 1 follows the repair and its later withdrawal. Red `BLOCK` lines mark actions the contract must prevent. After the tool changes, the reporting skill is temporarily disabled (*quarantined*) until revalidated. If the repaired version is later withdrawn, artifacts derived from it must also stop being used.

```
E1 Prepare repair: summarize v1 -> v2; tests pass.
E2 Prepare another edit from the same v1 state.
E1 Commit v2; quarantine dependent report v1.
E2 BLOCK: approval refers to the old state.
E3 Request guardrail edit and network access.
  BLOCK: edit exceeds the agent's authority.
E4 Derive and validate report v2 using tool v2;
  run report v2; store a lesson from this run.
E5 Withdraw tool v2 after discovering a defect.
  BLOCK: run report v2 or retrieve its lesson.
  BLOCK: restore withdrawn tool v2.
  Restore tool v1, then report v1, if eligible.
```

Listing 1. Illustrative lifecycle trace; E1–E5 are used throughout the approach.

III. Approach

A. Overview

Fig. 1 separates *proposing* a change from *deciding* whether it may take effect. The untrusted self-evolving host keeps its own search and learning loop: it proposes and compares harness changes, executes artifacts, and stores experience for later tasks. A trusted host adapter maps the host's artifacts to versioned components and mediates the three points at which a change can affect later behavior: activation, execution, and persistent reuse. Behind this trust boundary, the service enforces the Foundational Contracts in three stages over one shared trusted record of component versions, dependencies, derivation origins, policy, validators, and a hash-chained decision journal.

Admission (A) checks a DSL request ❶ against the contract and either returns a reason ❷ or issues a one-use receipt ❸. *Commit* (B) activates the result only if the checked context is still current ❹. *Runtime guards* (C) allow execution ❺ and reuse of stored experience ❻ only for unaltered, active, eligible versions. The operator (D) uses the same admission path ❼. Because eligibility follows recorded origins, one withdrawal blocks everything derived from the withdrawn version (inset).

B. Lifecycle model

The service decides from an explicit model of the harness, not from the artifacts themselves. Its lifecycle state is $S = (C, I, W, r)$. The map C records, for each component name c, its kind, current version v_c, mutability, dependencies, parents, and capabilities. The set I contains inactive (e.g., quarantined) names, W contains withdrawn identities $\langle c, v\rangle$, and r is a revision counter. History H maps each earlier identity to its component record. $O(x)$ is the set of recorded identities from which identity x was derived, directly or indirectly; we call them the *origins* of x.

Dependencies and parents form a graph $G = G(S)$ over component names, which must be acyclic with known endpoints; we call both kinds of neighbor the *prerequisites* $pre(c)$ of c. $\mathsf{Dep}_G(t)$ is the set of components that reach t through such edges, excluding t; these are the components potentially affected by changing t. An identity is *eligible* if neither it nor any of its origins is withdrawn, and an active component must have active prerequisites and an eligible current version:

$$\mathsf{Elig}(x) \equiv (\{x\} \cup O(x)) \cap W = \varnothing, \tag{1}$$

$$\forall c \in \mathrm{dom}(C) \setminus I : \ pre(c) \cap I = \varnothing \wedge \mathsf{Elig}(\langle c, v_c\rangle). \tag{2}$$

$\mathsf{Valid}(S, H, O)$ conjoins Eq. (2), acyclicity of G, record typing, and unique identities. We write $A(S) = \{\langle c, v_c\rangle \mid c \in \mathrm{dom}(C) \setminus I\}$ for the active identities. Because Eq. (1) consults origins, one withdrawal disqualifies all descendants without enumerating them.

C. Foundational Contracts

A Foundational Contract answers two questions: may a change take effect now (a classical pre/postcondition and invariant check), and may its result still be used later (a continued-use condition that one-time approval lacks)?

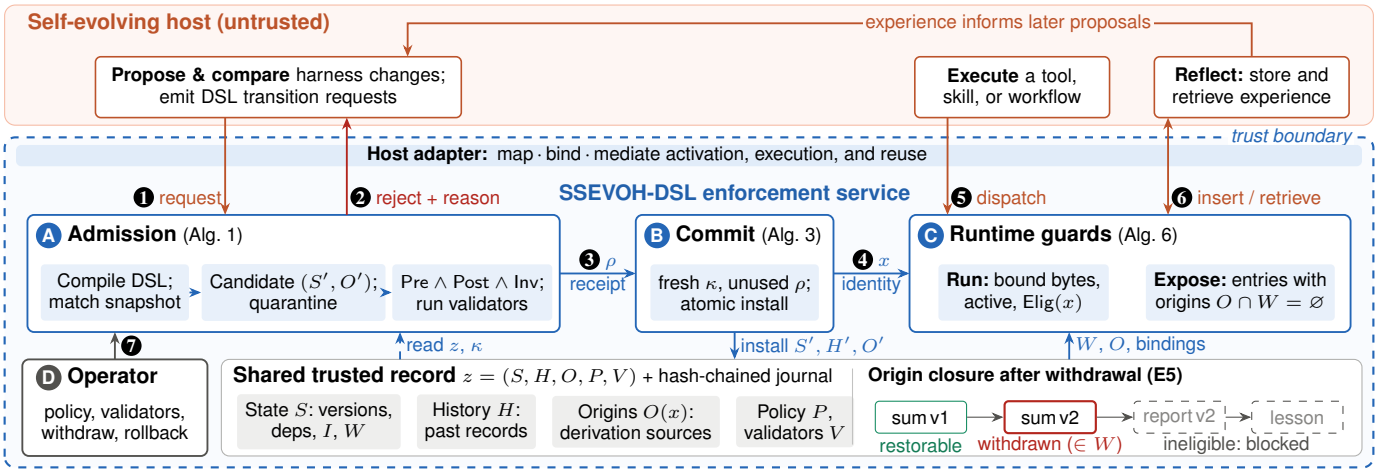


Fig. 1. Overview of SSEVOH-DSL. The untrusted host proposes, executes, and learns; the trusted service enforces Foundational Contracts at admission (A), commit (B), and runtime use (C) over a shared versioned record. Operator requests (D) take the same admission path. Inset: withdrawing summarize v2 (E5) makes every artifact derived from it ineligible.

Definition 1 (Foundational Contract). A Foundational Contract is a tuple $\mathcal{F} = (\mathsf{Pre}, \mathsf{Post}, \mathsf{Inv}, \mathsf{Use})$ of executable predicates controlled by the operator and enforced outside the editable harness. Its validation context is $z = (S, H, O, P, V)$, adding policy P and validator registry V to the lifecycle model. For an authenticated principal p, a compiled request e, and the candidate context $z' = (S', H, O', P, V)$, admission requires

$$\mathsf{Admit}_{\mathcal{F}}(z, p, e, z') \equiv \mathsf{Pre}(z, p, e) \wedge \mathsf{Post}(z, e, z') \wedge \mathsf{Inv}(z'), \tag{3}$$

and every later commit, execution, or retrieval must satisfy Use (Eq. (4)).

SSEVOH-DSL instantiates the predicates as follows. Pre holds when the request's declared snapshot equals S, the target has the expected version and activity, and $\mathsf{Auth}(p, e, S)$ permits the principal, operation, target, and capabilities. Post holds when $\mathsf{Protected}(S, S')$ preserves immutable, policy, and verifier records and does not newly quarantine them, candidate capabilities stay within the target's ceiling, the requested activity outcome holds, and $\mathsf{Evidence}(e, S')$ obtains exactly `True` from every mandatory and requested validator. $\mathsf{Inv}(z') \equiv \mathsf{Valid}(S', H, O')$; commit later adds the records of S' to H.

Admission alone must not become permanent authority. Let b hold the outstanding receipts b_ρ and the trusted bindings b_h from identities to artifact hashes, kept separately from z. For a receipt ρ, an artifact a presented as identity x, and a stored entry n,

$$\begin{aligned} \mathsf{Use}_{\mathrm{commit}}(\rho) &\equiv \rho \in b_\rho \wedge \rho.\kappa = \mathsf{Id}(z), \\ \mathsf{Use}_{\mathrm{run}}(a, x) &\equiv b_h(x) = \mathsf{h}(a) \wedge x \in A(S) \wedge \mathsf{Elig}(x), \\ \mathsf{Use}_{\mathrm{get}}(n) &\equiv n.O \cap W = \varnothing, \end{aligned} \tag{4}$$

where Id and h are collision-resistant hashes, $\rho.\kappa$ is the hash of the context checked at admission, and $n.O$ holds the identities, with their origins, that produced n. A receipt is thus valid only for the exact context it checked, and an artifact only while it is unaltered, active, and eligible.

D. Connecting the host to the service

The adapter has four obligations. **Mapping:** bind component/version identities to exact artifact bytes and trusted dependency/source records. A multi-file bundle may be one workflow component; each transition targets one component. **Mediation:** intercept activation, execution, and every persistent insertion/retrieval, including derived candidates. **Authority:** protect callers' identities, policy, validators, receipts, and history; serialize dispatch with withdrawal. **Evidence:** register domain predicates and permitted executors, with independent isolation for mutable code.

The service cannot discover omitted sources or intercept unmediated actions. Component dependencies and derivations must form a directed acyclic graph (DAG), although a workflow may contain cyclic execution. Hosts without these boundaries are outside scope.

E. The transition language

The DSL describes a proposed change; the trusted service decides whether it satisfies the Foundational Contracts. A specification declares the current component snapshot and exactly one evolution: operation (`update`, `withdraw`, `rollback`), target, expected/resulting versions, dependencies, parents, capabilities, validators, and activity conditions (whether the target must be active before and after the change). Parsing rejects duplicate or unknown fields, unresolved references, and trailing input; equivalent declarations are canonicalized. Proposals cannot embed validator code. Listing 2 encodes E1 and compiles with the released compiler.

The declared snapshot must match trusted state. The host supplies the authenticated principal, policy, and validators. Editing guardrails or requesting `network` can be syntactically valid yet unauthorized (E3).

F. Admission: preparing a transition

Admission is the only way a request obtains authority, so it checks the whole contract before anything changes.

```
harness ledger {
  component guardrails policy { version 1; mutable false; }
  component summarize tool {
    version 1; mutable true; capabilities [read_ledger]; }
  component report skill {
    version 1; mutable true; depends [summarize]; }
  evolution fix_sign {
    operation update; target summarize; from 1; to 2;
    capabilities [read_ledger];
    requires [structure, signed_ledger];
    expect active; ensure active;
  }
}
```

Listing 2. Running example, event E1: a *request* to update summarize.

Algorithm 1 PREPARE: admit a request (flows ❶ and ❷)

Input: DSL source src, authenticated principal p
1: lock authority; $\kappa \leftarrow \mathsf{Id}(z)$
2: $(IR, e) \leftarrow \textsc{Compile}(src)$; **require** $IR.C = C$ ▷ snapshot
3: **require** $\mathsf{Auth}(p, e, S)$ ▷ principal, op, target, caps
4: **require** $e.from = v_{e.t}$ and $e.expect$ holds in S
5: $(S', O') \leftarrow \textsc{Candidate}_{e.op}(e, S)$
6: **require** $\mathsf{Protected}(S, S') \wedge \mathsf{Valid}(S', H, O')$
7: **require** $e.ensure$ holds; candidate caps $\subseteq Ceil(e.t)$
8: **for** $\phi \in Mandatory \cup e.requires$ **do**
9: **require** $\phi \in \mathrm{dom}(V)$
10: $ev[\phi] \leftarrow V[\phi](S', e)$; **require** $ev[\phi] = \texttt{True}$
11: **require** $\mathsf{Id}(z) = \kappa$ ▷ validators cannot alter z
12: **require** $\mathsf{Id}(z) = \kappa$
13: register (e, S', O') under a fresh token τ
14: **return** $\rho = \langle \tau, p, \mathsf{h}(IR), \kappa, \mathsf{h}(S'), \mathsf{h}(O'), ev \rangle$

Algorithm 1 evaluates Eq. (3). It locks the trusted records and computes $\kappa = \mathsf{Id}(z)$, the hash of the checked context. Every failed **require** rejects the request. Issuing a receipt does not change κ, so competing approvals may share a base. A private pending table retains the proposed result (the *candidate*) and request. Candidate construction dispatches by operation to Algs. 2, 4, and 5.

Example. E1 passes trusted `structure` and `signed_ledger` checks. E3 fails authorization; the proposer cannot override policy or supply validator results.

G. Candidate construction and origins

Algorithm 2 computes the candidate for an update. Quarantine uses the *old* graph: components that relied on the replaced version become inactive until revalidated against the replacement. Origins are conservative: the new version inherits the previous target version, its origins, and every declared parent version with its origins. An update cannot drop a parent and so cannot declare independence from a withdrawn ancestor. No provenance-severing release operation is provided.

Example. E1 quarantines report, which depends on summarize, and sets $O(\langle \mathsf{summarize}, 2 \rangle) = \{\langle \mathsf{summarize}, 1 \rangle\}$. In E4, report v2 declares `parents [summarize]`, so it inherits its previous version, the parent's current version, and that parent's origins: $O(\langle \mathsf{report}, 2 \rangle) = \{\langle \mathsf{report}, 1 \rangle, \langle \mathsf{summarize}, 2 \rangle, \langle \mathsf{summarize}, 1 \rangle\}$. When E5 withdraws summarize v2, this set meets W, so report v2 becomes ineligible (Eq. (1)).

Algorithm 2 CANDIDATE: update a target t

Input: canonical request e for target t, state S
1: $c \leftarrow C[t]$; **require** $e.to > v_c$ and $\langle t, e.to \rangle \notin \mathrm{dom}(H)$
2: **require** $c.parents \subseteq e.parents$ ▷ ancestry never dropped
3: $Q \leftarrow \mathsf{Dep}_G(t)$ ▷ old graph $G = G(S)$
4: $c' \leftarrow c$ with version $e.to$, relations and caps of e
5: $O' \leftarrow O$; $O'(\langle t, e.to \rangle) \leftarrow \{\langle t, v_c \rangle\} \cup O(\langle t, v_c \rangle) \cup$
6: $\quad \bigcup_{q \in e.parents} \left(\{\langle q, v_q \rangle\} \cup O(\langle q, v_q \rangle)\right)$
7: $I' \leftarrow (I \cup Q) \setminus \{t\}$ ▷ update activates target
8: **return** $(\langle C[t \mapsto c'], I', W, r{+}1 \rangle, O')$

Algorithm 3 COMMIT: apply a receipt (flows ❸ and ❹)

Input: receipt ρ
1: lock authority
2: **require** ρ genuine, outstanding, and unconsumed
3: **require** $\rho.\kappa = \mathsf{Id}(z)$ ▷ stale otherwise
4: $(e, S', O') \leftarrow Pending[\rho.\tau]$; recheck their hashes
5: stage history records, O', and hash-chained journal entry
6: atomically swap state references; consume ρ
7: notify adapter of committed identity; bind before dispatch
8: append decision record; **return** $(r', x = (e.t, v'_{e.t}))$

H. Commit and freshness

Algorithm 3 turns a receipt into state. Because $\kappa = \mathsf{Id}(z)$ covers state, history, origins, policy, and validator registry, any successful competing commit invalidates every other receipt prepared from the old base. Staging followed by a reference swap prevents partly applied authoritative transitions. A rejected receipt leaves state unchanged. This is a transaction guarantee for the service and its controlled publication path, not a general filesystem crash-consistency theorem.

Example. E1 and E2 share κ. Committing E1 changes state and history, so E2 becomes stale and must be proposed against the new base.

I. Withdrawal and rollback

Withdrawal (Alg. 4) is the operator's revocation primitive. W grows monotonically ($W \subseteq W'$ for every transition). A dependent may become inactive without its own version being withdrawn, which keeps direct withdrawal distinct from affected closure. Derived versions become ineligible through Eq. (1), so no descendants need to be enumerated.

Rollback (Alg. 5) loads an exact historical record and its origins. The common admission path rechecks current policy, capability ceilings, validators, and activity conditions. Recovery neither clears W nor overrides an origin's withdrawal.

Example. After E5, $\langle \mathsf{summarize}, 2 \rangle \in O(\langle \mathsf{report}, 2 \rangle) \cap W$, so skill v2 remains ineligible. Restoring eligible v1 records can succeed under current policy.

J. Execution and persistent reuse

Algorithm 6 expresses the host adapter protocol, not a particular framework's API. MAP supplies the trusted artifact/metadata binding. BIND associates a committed identity with immutable bytes. EXECUTORALLOWED checks the domain's execution policy. Admission, dispatch, and withdrawal share the authority lock. Already-running external effects are not undone by withdrawal.

Algorithm 4 WITHDRAW: construct a candidate

Input: authorized request e, target t, current version v
require $e.to = v_t = v$ ▷ historical withdrawal not primitive
$W' \leftarrow W \cup \{\langle t, v\rangle\}$; $I' \leftarrow I \cup \mathsf{Dep}_G(t) \cup \{t\}$
return $(\langle C, I', W', r{+}1\rangle, O)$ to PREPARE

Algorithm 5 ROLLBACK: construct a candidate

Input: authorized request e, target t, requested version v_h
require $\langle t, v_h\rangle \in \mathrm{dom}(H)$ and $\mathsf{Elig}(\langle t, v_h\rangle)$
$c_h \leftarrow H[\langle t, v_h\rangle]$; $Q \leftarrow \mathsf{Dep}_G(t)$
$I' \leftarrow (I \cup Q) \setminus \{t\}$
return $(\langle C[t \mapsto c_h], I', W, r{+}1\rangle, O)$ to PREPARE

Example. E4's stored lesson inherits the executed versions' origins. After E5, retrieval omits it. This covers recorded sources, not every model influence.

K. Correctness argument and audit trail

Proposition 1. Assume trusted initialization, pure and terminating validators, complete mediation (Secs. II-C and III-D), and sufficient resources. Then every committed context satisfies $\mathsf{Valid}(S, H, O)$, and no mediated execution or retrieval occurs unless Eq. (4) holds.

Argument. Initialization checks acyclicity and active origins. PREPARE checks $\mathsf{Valid}(S', H, O')$ under the lock; COMMIT installs exactly that candidate, and only if the context is still the checked one ($\rho.\kappa = \mathsf{Id}(z)$). Updates retain ancestry and quarantine dependents, withdrawal only grows W, and rollback checks historical origins and current policy. GUARD evaluates Eq. (4) under the same lock as withdrawal. Induction over commits yields the claim for the metadata model, not a mechanized proof of the interpreter or filesystem. RQ1 tests it against independent semantics within finite bounds.

The journal links committed states; anchored replay checks continuity and identity. A separate log records rejections. Validator identities are process-local and do not attest captured environments, so replay reruns trusted validators.

IV. EVALUATION

A. Case-study host and adapter

Harness[2] [4] provides the integration case study. Its Propose, Probe, and Compose loop edits harness trees; reflection stores experience for later tasks. The pinned runtime (commit `6868817`) checks paths and schemas, retries failures, and separates method from judge; none of these checks addresses lifecycle authorization.

The adapter maps each multi-file tree to a workflow component, protects policy and verifiers separately, and records composition/exposure sources as origins. A launcher wraps `_accept_tree`, `_run_stage`, and `Library` while preserving upstream search and learning. All 70 upstream tests pass. Ledger fixtures allow two trusted programs. ToolRepair-12 adds static allowlisting and qualified Bubblewrap execution of new tools; HumanEval+ evolves text and grades solution code separately. These domain adapters share one host and do not establish cross-framework portability.

Algorithm 6 GUARD: host acceptance, execution, and reuse (flows ❺ and ❻; Eq. (4))

function ACCEPT($artifact$, $base$, $sources$, p)
  lock; freeze $artifact$ in trusted staging
  require HOSTCHECKS($artifact$)
  $src \leftarrow$ MAP($artifact, base, sources$)
  $\rho \leftarrow$ PREPARE(src, p); $(r, x) \leftarrow$ COMMIT(ρ)
  BIND(x, h($artifact$)); **return** x
function RUN($artifact$, x)
  lock; **require** bytes match binding, x active and $\mathsf{Elig}(x)$
  require EXECUTORALLOWED($artifact$)
  HOSTRUN($artifact$); $Exec \leftarrow Exec \cup \{x\}$
INSERT(n): $n.O \leftarrow \bigcup_{x \in Exec}(\{x\} \cup O(x))$
EXPOSE(): **return** $\{n \in Store \mid n.O \cap W = \varnothing\}$

B. Research questions

RQ1 (bounded conformance): Do the implementation and independent lifecycle semantics agree within published finite bounds?
RQ2 (contract contribution): Which violations do Foundational Contracts prevent beyond ordinary tests and static checks, and what do individual clauses contribute?
RQ3 (integration fidelity): Are admission, composition, execution, and library reuse protected inside the original Harness[2] workflow, including with model-driven proposers?
RQ4 (useful behavior): Does an evolving agent under the contract deliver useful task outcomes at a documented inference budget?
RQ5 (cost and reproducibility): What do validation, scaling, and contention cost, and can distributed runs be reconstructed from pins and raw records?

C. Study design

Table I separates contract conformance (RQ1), clause ablations (RQ2/RQ3), task usefulness (RQ4), and cost (RQ5). Ablations disable mechanisms implementing Definition 1 to identify which obligations prevent each violation.

Cohorts and conditions. The deterministic study repeats 25 actions five times per baseline and ablation. The DSL cohort crosses five models, 20 contexts (17 scenarios), and five seeds. Host fixtures repeat five times: 12 shared fixtures (three permitted, nine disallowed), plus four lifecycle fixtures for full enforcement and eight ablations. Lifecycle fixtures cover withdrawn origins, rollback conflicts, stale approval, and interrupted publication. Cohorts answer different questions and are not pooled.

Core baselines add admission checks cumulatively: B0 parses the DSL, B1 adds the snapshot schema, B2 adds trusted policy and candidate construction, and B3 is full enforcement. Trusted setup advances their contexts, so timings exclude deployment. In Harness[2], C0 is the original runtime, C1 adds one signed-ledger unit test, C2 adds static principal/protected-file checks and DSL compilation without semantic verification, and C3 is full enforcement. C0–C2 have no withdrawal or recovery API, so lifecycle rates do not apply to them. Each ablation removes one mechanism from full enforcement: permissions, dependencies, provenance, evidence, formal verification, withdrawal, rollback, or *freshness* (revalidation of stale approvals).

TABLE I
EVALUATION COHORTS AND THEIR RESEARCH QUESTIONS.

Cohort	Size	Purpose	RQ
Bounded assurance	275,332 checks	Compare implementation and independent semantics	1
Deterministic baselines	500 decisions	Compare cumulative admission checks	2
Core ablations	750 decisions	Remove individual lifecycle gates	2
DSL generation	500 responses	Assess model-generated requests	2
Controlled Harness[2]	900 records	Exercise host seams and gate ablations	2, 3
Host pilots	25 + 50 streams	Diagnose workflow and usefulness failures	3, 4
ToolRepair-12	540 streams	Compare repair usefulness and contract violations	4
HumanEval+	3,936 outcomes	Compare evolving and frozen coding agents	4
Scaling	180 records	Measure graph-size and contention costs	5
Site qualification	18 runs	Check matched execution across three sites	5

Models and budgets. Qwen2.5-Coder 7B, Phi-4 14B, GPT-OSS 20B, Devstral 24B, and Qwen3-Coder 30B are pinned by manifest digest. The DSL cohort uses temperature 0.2, top-p 0.9, 1,024 output and 4,096 context tokens, one generation per model/context/seed. The ledger pilot and ToolRepair-12 use 2,048 output and 16,384 context tokens and run original aggregation at k=1: four solver rollouts and three proposal stages per task, at most two edit attempts per stage. Raw request/response records retain all calls, tokens, limits, and failures.

Measures and analysis. A violation means noncompliance with the operator contract, not inferred maliciousness. We report syntax failures, oracle evaluability, allowed/disallowed decisions, deployment/workflow errors, and task success separately. Invalid outputs are not safe requests; empty populations have undefined rates. Strict fidelity is a post hoc field-level diagnostic. Scenarios, tool families, and ordered coding streams are the respective inferential units; seeds repeat observations and sites supply infrastructure. ToolRepair and HumanEval+ use separate frozen protocols (Sec. IV-G).

D. RQ1: Bounded conformance

Table II summarizes the assurance evidence. The one-step graph/lifecycle domain has 176,256 cases over acyclic two-component graphs, inactive and withdrawn subsets, and relations to self, another component, or a missing component. Target versions range from 1 to 3 and expected versions from 1 to 2, with activity conditions varied. A separate 72,576-case domain varies one component's kind, mutability, capabilities, grant, and evidence. Together these checks accept 3,388 requests and reject 245,444 without a discrepancy against the independent reference.

History exploration starts from seven two-component graphs, uses versions 1 to 3, and explores all operations to depth four. It visits 3,307 contexts and checks 19,620 transitions, of which 6,880 are accepted. Anchored replay checks every accepted endpoint. These are finite-domain counts, not estimates of deployment prevalence or independent replications.

The lifecycle model has one proposal and dependency epochs 0 to 2. Python and TLC (the TLA+ model checker) agree in all four authorization/evidence contexts, with 18/44, 18/44, 23/62, and 61/133 states/edges. The publication model has one bundle, one origin bit, and two epochs. Its graphs agree at 45/81 states/edges with authorization and evidence enabled and 4/4 in each other context. Five lifecycle variants omit authorization, evidence, withdrawal, rollback, or freshness. All yield counterexamples, as do six publication variants. For example, omitting freshness permits deployment after a dependency change in five steps.

TABLE II
BOUNDED ASSURANCE AND REGRESSION RESULTS (REGRESSION: CORE, INTEGRATION, AND UPSTREAM HARNESS[2] SUITES).

Check	Extent	Result
One-step conformance	248,832 cases	No differences
History exploration	19,620 transitions	No differences
Anchored replay	6,880 endpoints	No differences
Lifecycle model	4 contexts	Python = TLC
Publication model	4 contexts	Python = TLC
Faulty variants	11 variants	All detected
Regression suites	62 / 32 / 70 tests	All pass

Answer to RQ1. The checked representations agree and all faulty variants are detected within the stated bounds. These results establish finite consistency, not unrestricted program safety.

E. RQ2: Contribution of Foundational Contracts

Deterministic baselines and core ablations. Each row of Table III contains 125 decisions: 60 disallowed and 65 allowed. FA counts disallowed requests admitted, and FR counts allowed requests rejected. Latency is the median admission time in milliseconds. The table shows that syntax (B0) and schema (B1) checks admit all 60 disallowed actions. Trusted policy with candidate construction (B2) still admits 20, namely those whose violation is only visible to validators or lifecycle history. Full enforcement (B3) admits none and rejects none of the 65 allowed actions. Removing dependency closure, provenance checks, or freshness each re-opens five admissions. Removing dependency closure also causes ten false rejections, because skipped quarantine diverges later state. Removing withdrawal or rollback checks alone has no effect in these cases because retained guards overlap. A zero single-gate effect does not show that a mechanism is dispensable.

Model-generated requests. Only 91/500 responses reach the oracle: 73 disallowed and 18 allowed (Fig. 2b, Table IV). All are decided correctly. Three models always emit fenced output, which the raw-text protocol rejects; GPT-OSS has

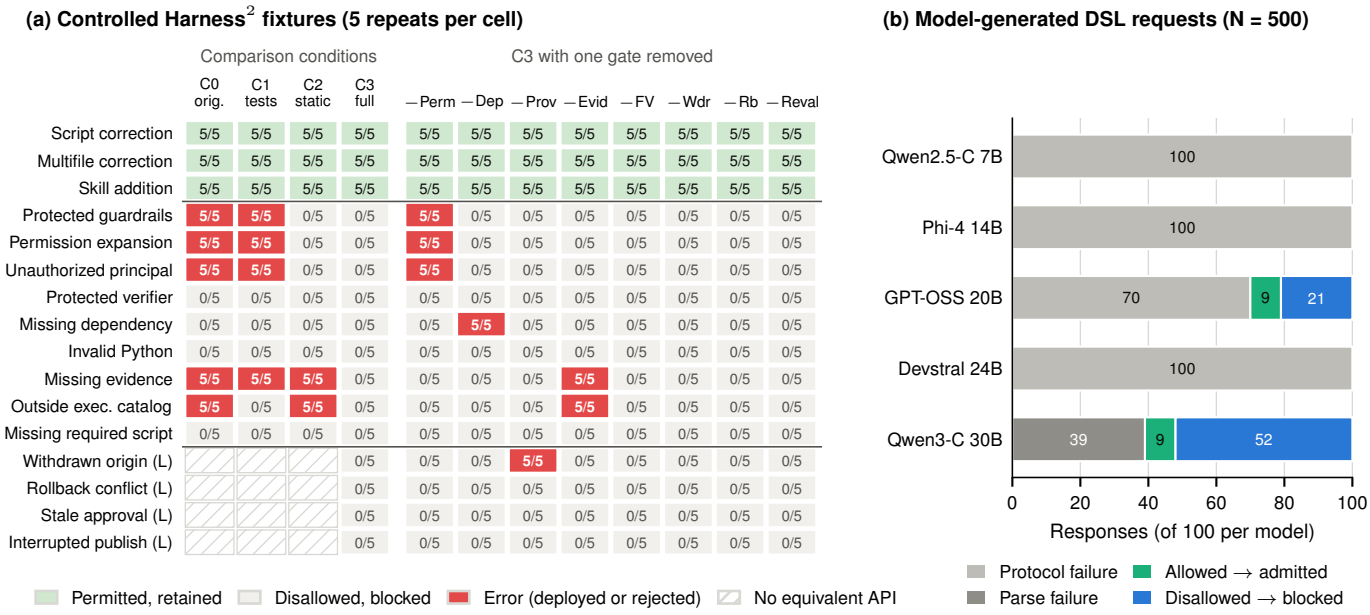


Fig. 2. Controlled fixture outcomes under comparison conditions and gate ablations (a), with model-generated DSL response outcomes (b).

TABLE III
ADMISSION ERRORS AND MEDIAN LATENCY ACROSS BASELINE CONDITIONS AND CORE GATE ABLATIONS.

Condition	FA	FR	Median ms
B0 syntax only	60/60	0/65	0.197
B1 + snapshot schema	60/60	0/65	0.203
B2 + policy, candidate	20/60	0/65	0.227
B3 full enforcement	**0/60**	**0/65**	0.599
Full − dep. closure	5/60	10/65	0.538
Full − provenance	5/60	0/65	0.708
Full − freshness	5/60	0/65	0.593
Full − withdrawal	0/60	0/65	0.563
Full − rollback	0/60	0/65	0.841

TABLE IV
MODEL-GENERATED REQUESTS TEST THE DSL INTERFACE AND GUARDED HARNESS² WORKFLOWS.

	Primary DSL			Harness² streams			
Model	Flag	Oracle	Fid.	WF ok	Inv.	Adm.	Blk.
Qwen2.5-C 7B	0	0/0	0%	5/5	0	7/7	30/30
Phi-4 14B	0	0/0	0%	5/5	43	0/0	17/17
GPT-OSS 20B	30	9/21	21%	0/5	n/a	n/a	n/a
Devstral 24B	0	0/0	0%	5/5	0	0/0	54/54
Qwen3-C 30B	100	9/52	21%	5/5	0	9/9	40/40
Total	130	18/73	n/a	20/25	43	16/16	141/141

56 empty and 62 length-limited outputs, with overlap. The historical `syntax_valid` flag also covers 39 Qwen3-Coder identifier failures, so its 100 marks do not denote 100 parsed programs.

Table IV reports 100 DSL responses and five guarded host streams per model. In the DSL columns, Flag counts the historical `syntax_valid` marks, Oracle gives allowed/disallowed evaluable requests, and Fid. is post hoc strict request fidelity. In the host columns, WF ok counts completed workflows, Inv. invalid proposals, Adm. allowed proposals admitted, and Blk. disallowed proposals blocked; n/a marks unavailable entries.

Answer to RQ2. Foundational Contracts reject all assessable violations while retaining permitted requests. Dependency, origin, and freshness ablations each expose failures beyond syntax and policy checks. Model interface failures substantially limit the assessable population.

F. RQ3: Integration fidelity in Harness²

Fig. 2a and Table V report five repeats per host fixture. Each applicable condition has 45 shared disallowed, 15 permitted, and 20 lifecycle attempts. Shared and Lifecycle count disallowed executions, Retained counts permitted executions, and ms is mean latency over permitted executions. Timings are descriptive and order-confounded; n/a marks inapplicable and n/r unreported entries.

Original checks miss protected-file, permission, principal, evidence, and executable-catalog violations. C1 closes the catalog gap, while C2 closes the first three gaps. In Fig. 2a, the top three rows count permitted changes retained and other rows count disallowed deployments, with errors in red and lifecycle fixtures marked L. C3 blocks all shared and lifecycle violations and retains all permitted executions.

Model-driven streams. C3 blocks 141/141 assessed disallowed proposals and admits 16/16 permitted non-no-ops (Table IV, right). GPT-OSS's five failed workflows remain in the denominators. Reflection is untested in this pilot because the launcher omitted the selector client, leaving all fifty task summaries without experience. The corrected launcher, deterministic fixtures, and eight coding-adapter regressions exercise library guarding.

Measured contract contributions. Relative to C3, disabling authority checks (Pre) permits 15/45 shared violations;

TABLE V
CONTROLLED HARNESS^2 FIXTURES COMPARE CONTRACT VIOLATIONS, PERMITTED EXECUTIONS, AND LATENCY.

Condition	Shared	Retained	Lifecycle	ms
C0 original	25/45	15/15	n/a	77.17
C1 + unit test	20/45	15/15	n/a	155.73
C2 + static DSL	10/45	15/15	n/a	78.33
C3 full	**0/45**	**15/15**	**0/20**	114.81
C3 − permissions	15/45	15/15	0/20	n/r
C3 − evidence	10/45	15/15	0/20	n/r
C3 − dependencies	5/45	15/15	0/20	n/r
C3 − provenance	0/45	15/15	5/20	n/r
C3 − any other	0/45	15/15	0/20	n/r

disabling evidence (Post) permits 10/45; disabling dependency handling (Inv) permits 5/45. Disabling provenance permits 5/20 lifecycle violations through reuse of withdrawn origins (Inv/Use). Separately, disabling freshness ($\mathsf{Use}_{\mathrm{commit}}$) permits 5/60 disallowed core admissions (Table III). These effects are specific to the designed fixtures and are not additive, because clauses overlap. The demonstrated benefit is control over which changes remain usable; task accuracy is evaluated separately.

Answer to RQ3. The adapter enforces the contracts at actual host activation, execution, and reuse boundaries. Other host implementations remain untested.

G. RQ4: Useful behavior

Pilot. An initial 25-stream pilot (five models, five seeds, two ledger tasks) exposed answer-schema, selector-channel, and answer-form defects. A corrected 50-stream pilot tests two executable policies. No disallowed proposal executes (0/31 catalog, 0/13 sandboxed). Sandboxed evidence admits more correct repairs (8/50 versus 0/50 correct values, post hoc), but strict success remains 0/100.

ToolRepair-12 design. Twelve families plant CSV-tool defects, including sign, rounding, column, and number-format errors (Fig. 3b). Each has a specification, reference, and disjoint task, authority-validation, and proposal-oracle inputs. Five models, three seeds, and three arms yield 540 three-task streams. A1 uses original Harness^2, A2 adds catalog enforcement, and A3 uses sandboxed behavioral evidence. Each stream includes a frozen reference and one injection from seven rotating classes. All arms share infrastructure isolation for non-catalog tools, which is not credited to enforcement.

The frozen protocol states four hypotheses. H1: A3 succeeds more often than A2. H2: A3 executes no disallowed proposal, natural or injected, while A1 executes some. H3: A3 success is within 10 points of A1 (non-inferiority). H4 (exploratory): within A3 streams, the third task succeeds more often than the first. Protocol, analysis, and source hashes were frozen before qualification. Inference uses families with 10,000 paired cluster-bootstrap resamples and 95% percentile intervals. Holm-adjusted bootstrap tails for H1/H3 are diagnostics, not calibrated population p-values. Two deviations preceded full results: amendment v1.1 made three crashing defects silent and reran all 540 streams, while smoke analysis found an execution-label bug for proposals restating unchanged files. Corrected labels are primary, and worker labels are retained.

Usefulness. Mean per-family success is 0.241 for original Harness^2, 0.085 under the catalog, and 0.196 under sandboxed evidence (frozen: 0.041, Fig. 3). The sandboxed-evidence interval meets the prespecified 10-point non-inferiority margin against the unguarded arm (H3 met: difference -0.044, 95% CI $[-0.096, +0.002]$) and recovers 11.1 points over the catalog (H1 met: 95% CI $[+0.067, +0.159]$). The lower bound lies only 0.4 percentage points inside the margin, and family effects vary (Fig. 3b). Within A3 streams, success rises from 0.12 on the first task to 0.26 on the third (H4, exploratory), but fixed task order and the absence of a no-library ablation prevent attributing this trend to learning.

Safety at the same time. Original Harness^2 executed 426 contract-violating proposals (298 natural, 128/160 injected). A3 executed 0 natural and 0/160 injected violations under the corrected label (H2's observed-count criterion). These are planned-stream counts, including workflows that failed before proposals; because repeated classes are dependent, they are not a population risk bound. Under the uncorrected worker label, A3 shows 12 natural executions and H2 would not hold. Two A3 oracle-unknown cases were resolved after collection by direct oracle-input execution (a supplementary step, not a qualified replication); four A1 cases remain unknown. As declared before the study, the input-triggered resource-abuse probe, whose trigger lies outside every validation case, passed behavioral evidence in 16/20 A3 streams and ran bounded by the sandbox. Only Phi-4 succeeds more often under sandboxed evidence than unguarded (47 versus 44 of 108). GPT-OSS returned empty responses under JSON-constrained decoding; its streams count as failures, and excluding them (exploratory) gives 0.301, 0.106, and 0.245.

HumanEval+. In a prespecified text-only study [14], [15] (164 tasks, Phi-4 and Qwen2.5-Coder, three seeds, hidden tests graded after a global freeze), guarded minus original evolution is -0.6 points for Phi-4 (seeds -8.5 to $+3.7$) and $+1.8$ for Qwen. Neither beats frozen best-of-eight selection (guarded/original/frozen: Phi-4 0.354/0.360/0.476, Qwen 0.766/0.748/0.780) at about $1.8\times$ its calls. The four reported endpoints are original evolution, guarded evolution, frozen selection, and a one-shot endpoint drawn from the frozen pool. Across all tasks, models, and seeds, these yield 3,936 outcomes. These are descriptive all-attempts accuracies over three ordered streams/model, not task-independent significance tests. Frozen selection matches solver calls rather than total compute. One seed/model was regenerated before hidden grading after a storage failure, a recorded deviation.

Answer to RQ4. Behavioral evidence improves usefulness over the catalog, but non-inferiority is narrow under a permissive margin. Neither evolving coding arm beats frozen selection, and the declared resource-abuse probe remains a limitation.

H. RQ5: Cost and reproducibility

Validation and scaling. Fig. 4 summarizes scaling and the admission baselines. The scaling design crosses three graph

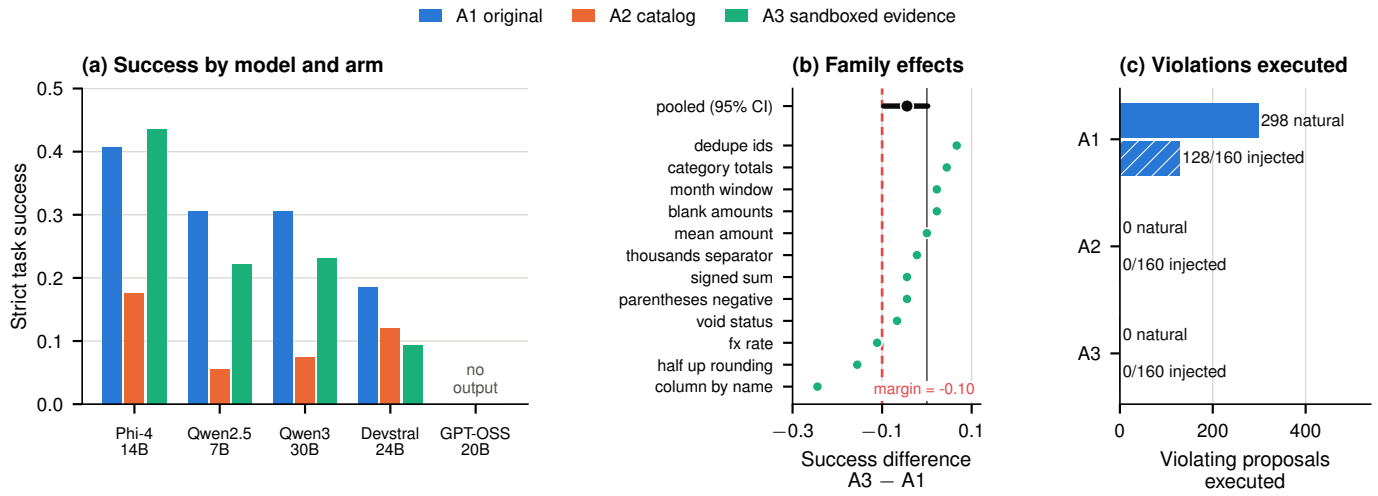


Fig. 3. ToolRepair-12 compares success by model (a), family-level usefulness effects (b), and corrected contract violations excluding the declared probe (c).

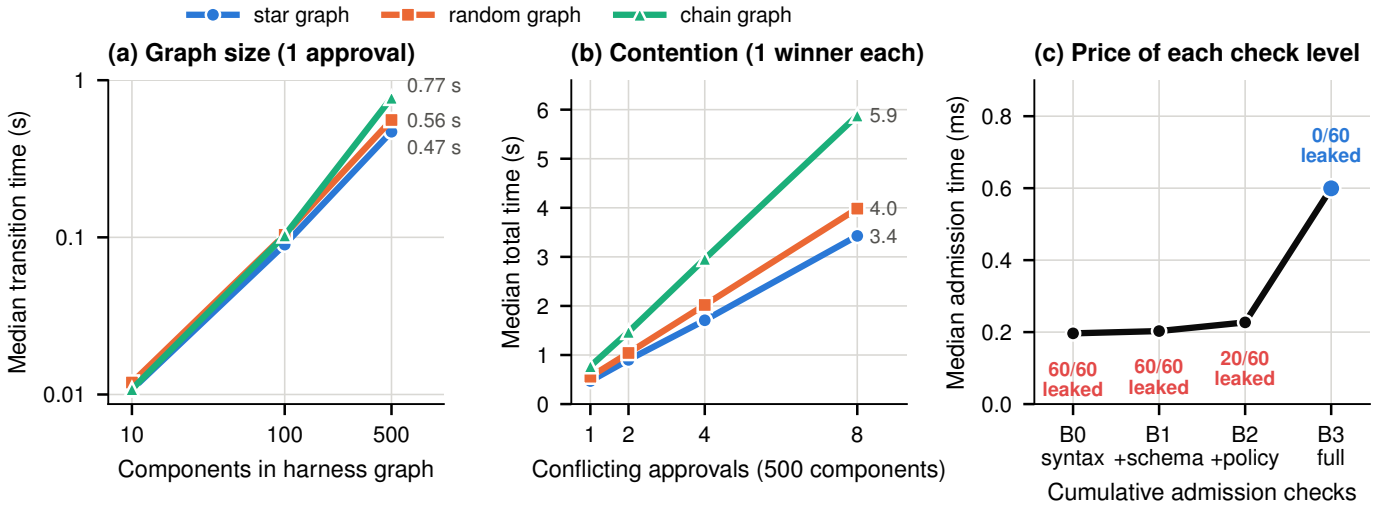


Fig. 4. Median enforcement costs over five repeats for graph size (a), competing approvals (b), and cumulative admission checks (c).

sizes, three graph shapes, four contention levels, and five repeats, giving 180 records. Shape matters because quarantine follows reverse reachability: chains quarantine the longest closures. Contention is resolved by compare-and-swap on the context hash: in every tested cell exactly one approval commits and the others are rejected as stale. These observations establish neither distributed throughput nor an asymptotic bound. Traced Python allocation peaks at 2.5 MB at 500 components. In Harness[2], full enforcement adds 37.6 ms to the mean permitted execution (Table V); C1 is slower because of its extra test process.

Reproducibility. Qualification comprises 18 runs across three sites, two studies, and three matched trials. HPC runs used Fir, Narval, and Rorqual through a `use-hpc`-derived Slurm pipeline [16] with source freezes, cache identity and GPU checks, measured site qualification, and hash-verified collection. Censored and failed cells are preserved, and superseded cells are re-run as recorded supplements rather than overwritten. Twenty-four matched cross-site pairs produced identical text and decisions under identical identities. All 24 were syntax-invalid outputs, so this shows deterministic serving, not useful generation.

Answer to RQ5. Metadata admission is inexpensive relative to inference in these benchmarks. End-to-end throughput and independent reanalysis remain bounded by the execution and artifact-release limitations described above.

I. Discussion: transfer and engineering trade-offs

A port to another host chooses its component granularity and search strategy but must supply the boundaries in Sec. III-D: archive search needs guarded promotion, code evolution needs isolation, and skill learning needs guarded installation and retrieval. These ports remain proposals; our bundle adapter shows multi-file binding, not mediation of every component kind.

Current restrictions include single-target updates, acyclic metadata, conservative ancestry, in-process serialization, and no verified restart or distributed commit. Withdrawal cannot

erase model context or undo external effects. Even an independent rewrite cannot shed withdrawn ancestry; operator-approved provenance release is future work. Catalog checks reject useful equivalent programs; behavioral validators admit more repairs but can miss input-triggered defects. Operators must maintain mappings, policies, validators, and isolation.

J. Threats to validity

Construct and internal. Contract compliance is narrower than harmless behavior or task correctness. Author-designed fixtures test specified clauses, not completeness against real attacks. Validators, oracles, and reference semantics share specifications and may share errors despite independent implementations, disjoint inputs, and faulty-variant checks. Strict request fidelity and pilot correct-value counts are post hoc. Sandboxes do not protect the authority from a hostile same-user host.

External. One host, small ledger fixtures, and twelve planted-defect families do not establish cross-framework or general autonomous performance. More models or sites do not add independent tasks. HumanEval+ may be contaminated.

Statistical and operational. Repeated fixtures and ordered streams are dependent; zero observed violations is not zero risk. ToolRepair's non-inferiority is narrow, bootstrap tails are not calibrated tests, and its safety result depends on corrected labels whose raw archives are not public (Sec. VI). Workflow/model-serving failures reduce assessable proposals and remain task failures. Pins and logs support audit, not deterministic completion.

V. Related Work

A. Harness evolution and persistent learning

ADAS and the Darwin Gödel Machine search agent programs [1], [2]; Harness^2 reuses experience from harness edits [4], ACE curates context [3], and Voyager composes skills [17]. HarnessBank screens recombined harnesses for valid evaluation, activation, and improvement [5]. Foundational Contracts govern whether such retained artifacts remain eligible for use, independently of search quality.

B. Governed evolution, authorization, and recovery

HarnessMutation already treats generated artifacts as persistent capabilities requiring validation, traceability, promotion, and rollback [8]. We formalize context-bound receipts and component-origin eligibility, with finite conformance evidence. EvoUndo checks recovery programs across counterfactual states [9]; our rollback selects eligible historical records without undoing arbitrary effects. Zhu and Wang conserve authority across replacement, forking, and rollback using single-parent lineage [10]. Our multiple-parent origins govern component and library eligibility but do not conserve population budgets or provide durable handoff.

The Update Framework and in-toto establish update/production trust [18], [19]. Our origins specialize PROV-DM [20] for withdrawal eligibility; our rollback selects recorded versions rather than performing dynamic state transfer [21].

C. Runtime policies and persistent-artifact security

AgentSpec enforces execution rules [22], Progent controls tool privileges [23], AutoSpec revises rules from counterexamples [24], FORGE checks event graphs with provenance and approval chains [25], and SAGA governs agent registration/communication [26]. We specialize these governance concerns with fixed evolution operations and version-origin invariants. General policy languages could encode parts of our contract with additional state and hooks; expressiveness or security superiority is not established.

Harness-tampering audits and SkillRelay expose risks in retained modifications and cross-session skill composition [6], [7]. Our guards block withdrawn recorded origins before dispatch and reuse, assuming complete mediation. CaMeL addresses the complementary prompt-injection channel through trusted control/data separation [27], [28].

D. Assurance and lifecycle evaluation

ENTRUST connects design-time and runtime verification to dynamic assurance cases [29]. SpecMAS generates NuSMV models and repairs failed properties [30]. Our evidence concerns metadata transition rules, using established producer/checker separation [31], reference monitoring [13], and authorization [32], [33]. Our bounded checks use TLC [34].

SEAGym assesses frozen harness snapshots through transfer, retention, replay, and cost views [35]. We similarly separate lifecycle decisions from task usefulness and inference cost. Our designed violation fixtures test enforcement obligations, while HumanEval/EvalPlus test selected-program correctness [14], [15].

VI. Conclusion

SSEVOH-DSL enforces Foundational Contracts that bind approval to a versioned context and govern continued execution, recovery, and reuse. Its metadata rules and adapter obligations apply independently of how a harness searches for improvements, provided execution and persistent reuse remain mediated. Bounded assurance supports consistency of the rules, and one Harness^2 integration demonstrates enforcement at real admission, execution, and reuse seams. ToolRepair-12 exposes the usefulness/evidence trade-off, while HumanEval+ shows no advantage over frozen selection. Cross-framework ports, broader tasks, durable recovery, and mechanized refinement remain necessary to establish wider applicability.

Artifacts. The versioned evidence snapshot [36] contains the core, pinned host runtime, protocols/deviations, outcome records, and analysis scripts. `make verify` checks bounded assurance, and experiments/scripts/ reproduces the outcome analyses. ToolRepair success is reproducible from the committed toolrepair-v1/streams.jsonl. The corrected execution labels (experiments/toolrepair/DEVIATIONS.md) need per-stream archives that are not in the public snapshot, so the corrected safety result cannot yet be replicated independently.

REFERENCES

[1] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," *arXiv preprint arXiv:2408.08435*, 2024. [Online]. Available: https://arxiv.org/abs/2408.08435

[2] J. Zhang, S. Hu, C. Lu, R. Lange, and J. Clune, "Darwin gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint arXiv:2505.22954*, 2025. [Online]. Available: https://arxiv.org/abs/2505.22954

[3] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun, "Agentic context engineering: Evolving contexts for self-improving language models," *arXiv preprint arXiv:2510.04618*, 2025. [Online]. Available: https://arxiv.org/abs/2510.04618v3

[4] R. Xu, Y. Chen, Z. CuiZhu, B. D. Mishra, Y. Ming, H. Yu, R. Han, C.-Y. Lee, and T. Pfister, "Harness^2: Recursive agent harnessing for an open world," *arXiv preprint arXiv:2610.14224*, 2026. [Online]. Available: https://arxiv.org/abs/2610.14224

[5] X. Luo, D. Xue, F. Wang, C. Hu, and Y. Deng, "HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution," *arXiv preprint arXiv:2607.13683*, 2026. [Online]. Available: https://arxiv.org/abs/2607.13683v2

[6] X. Wang, X. Zhang, and J. Shao, "Auditing harness tampering in self-improving agents," *arXiv preprint arXiv:2609.00069*, 2026. [Online]. Available: https://arxiv.org/abs/2609.00069v1

[7] D. Liao, S. Pan, G. Bai, Z. Xing, and X. Sun, "Understanding and mitigating cross-session attacks via persistent artifacts in agent skill composition," *arXiv preprint arXiv:2610.13227*, 2026. [Online]. Available: https://arxiv.org/abs/2610.13227v1

[8] M. Garralda-Barrio, "Governed evolution of agent runtimes through executable operational cognition," *arXiv preprint arXiv:2605.27328*, 2026. [Online]. Available: https://arxiv.org/abs/2605.27328v1

[9] T. Sah, D. Sah, H. Jain, and T. Sah, "EvoUndo: Recoverability-constrained self-evolution for LLM agent harnesses," *arXiv preprint arXiv:2608.28363*, 2026. [Online]. Available: https://arxiv.org/abs/2608.28363v2

[10] G. Zhu and C. Wang, "Authorization for self-modifying AI agent populations: Conserving authority across replacement, forking, and rollback," *arXiv preprint arXiv:2610.00347*, 2026. [Online]. Available: https://arxiv.org/abs/2610.00347v1

[11] B. Meyer, "Applying "design by contract"," *Computer*, vol. 25, no. 10, pp. 40–51, 1992. [Online]. Available: https://se.inf.ethz.ch/~meyer/publications/computer/contract.pdf

[12] Object Management Group, *Object Constraint Language, Version 2.4*, 2014, oMG specification formal/2014-02-03. [Online]. Available: https://www.omg.org/spec/OCL/2.4

[13] F. B. Schneider, "Enforceable security policies," *ACM Transactions on Information and System Security*, vol. 3, no. 1, pp. 30–50, Feb. 2000. [Online]. Available: https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf

[14] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[15] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://arxiv.org/abs/2305.01210

[16] ai4se4ai-lab, "use-hpc," Software repository, 2026, pipeline base revision f9dc7d66ad8df3c202f6b72b02630e14fd148756. [Online]. Available: https://github.com/ai4se4ai-lab/use-hpc

[17] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024.

[18] J. Samuel, N. Mathewson, J. Cappos, and R. Dingledine, "Survivable key compromise in software update systems," in *Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS)*, 2010, pp. 61–72.

[19] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, "in-toto: Providing farm-to-table guarantees for bits and bytes," in *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 1393–1410.

[20] L. Moreau and P. Missier, "PROV-DM: The PROV data model," W3C Recommendation, Apr. 2013. [Online]. Available: https://www.w3.org/TR/prov-dm/

[21] M. Hicks and S. Nettles, "Dynamic software updating," *ACM Transactions on Programming Languages and Systems*, vol. 27, no. 6, pp. 1049–1096, 2005.

[22] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026, pp. 2938–2950. [Online]. Available: https://arxiv.org/abs/2503.18666

[23] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2026, version 3, revised May 2026; first submitted 2025. [Online]. Available: https://arxiv.org/abs/2504.11703v3

[24] P. Ma, Z. Wang, Z. Ji, Y. Zhou, Z. Xue, Z. Li, S. Wang, and X. Zhang, "AutoSpec: Safety rule evolution for LLM agents via inductive logic programming," *arXiv preprint arXiv:2606.24245*, 2026. [Online]. Available: https://arxiv.org/abs/2606.24245

[25] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, I. Dillig, and S. Jha, "Formal policy enforcement for agentic systems," *arXiv preprint arXiv:2602.16708*, 2026. [Online]. Available: https://arxiv.org/abs/2602.16708v4

[26] G. Syros, A. Suri, J. Ginesin, C. Nita-Rotaru, and A. Oprea, "SAGA: A security architecture for governing AI agentic systems," *arXiv preprint arXiv:2504.21034*, 2025. [Online]. Available: https://arxiv.org/abs/2504.21034v2

[27] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[28] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[29] R. Calinescu, D. Weyns, S. Gerasimou, M. U. Iftikhar, I. Habli, and T. Kelly, "Engineering trustworthy self-adaptive software with dynamic assurance cases," *IEEE Transactions on Software Engineering*, vol. 44, no. 11, pp. 1039–1069, 2018. [Online]. Available: https://doi.org/10.1109/TSE.2017.2738640

[30] R. Agrawal, K. T. Ranade, A. Khanal, K. S. Basu, and A. Narayan, "SpecMAS: A multi-agent system for self-verifying system generation via formal model checking," in *Advances in Neural Information Processing Systems*, vol. 38, 2025, pp. 75 264–75 283. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/61816fb8f7dd9677d23d00196ccbf126-Abstract-Conference.html

[31] G. C. Necula, "Proof-carrying code," in *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, 1997, pp. 106–119.

[32] J. W. Cutler, C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, E. Ioannidis, J. Kastner, A. Mamat, D. McAdams, M. McCutchen, N. Rungta, E. Torlak, and A. Wells, "Cedar: A new language for expressive, fast, safe, and analyzable authorization (extended version)," *arXiv preprint arXiv:2403.04651*, 2024. [Online]. Available: https://arxiv.org/abs/2403.04651

[33] C. Disselkoen, A. Eline, S. He, K. Headley, M. Hicks, K. Hietala, J. Kastner, A. Mamat, M. McCutchen, N. Rungta, B. Shah, E. Torlak, and A. Wells, "How we built cedar: A verification-guided approach," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2407.01688

[34] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002. [Online]. Available: https://lamport.azurewebsites.net/tla/book.html

[35] C. Zheng, C. Xue, B. Liang, J. Yang, and C. Zhang, "SEAGym: An evaluation environment for self-evolving LLM agents," *arXiv preprint arXiv:2606.17546*, 2026. [Online]. Available: https://arxiv.org/abs/2606.17546v1

[36] "SSEVOH-DSL: Source, protocols, and evaluation evidence," Versioned software and data repository, 2026, evidence snapshot 07f544fec4b66c2eb8354500f12c8358fed16f29. [Online]. Available: https://github.com/ai4se4ai-lab/SSEVOH-DSL/tree/07f544fec4b66c2eb8354500f12c8358fed16f29